A few refresher exercises to get back in the saddle

Lino Galiana
Version of November 2, 2026

A chapter devoted to various exercises to review the basics of Python syntax and the objects used by the language.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Synthesis exercises 1
1.1 Exercice 1 1
1.2 Exercise 2 3
1.3 Exercise 3: strings 4
Informations additionnelles 5

1 Synthesis exercises

To make sure you are fluent on data structures and operations in `Python` for different problems, here is a series of exercises.

They require you to think hard about the appropriate data structure to answer - that's normal!

For once, the exercises are done directly on this page rather than via *notebooks*[1]. The correction will soon be available on the dedicated page.

1.1 Exercice 1

Exercice 1

You have at your disposal a list of quotations from the magnificent French literary heritage:

See the list of quotations that will be useful

Citation	Auteur
"Rien ne sert de courir ; il faut partir à point."	La Fontaine

[1]It is possible to run `Python` on the browser thanks to its implementation in WASM via Pyodide. This is not a recommended method when you need to perform big calculations, as in the next chapters. But since we are working on small examples here, this is quite practical to benefit from the interactivity enabled by the browser, notably the helpers in case ou are blocked.

Citation	Auteur
“Selon que vous serez puissant ou misérable, les jugements de cour vous rendront blanc ou noir.”	La Fontaine
“Heureux qui comme Ulysse a fait bon voyage”	Du Bellay
“L’homme est né libre, et partout il est dans les fers.”	Rousseau
“Parce que c’était lui, parce que c’était moi.”	Montaigne
“La première fois qu’Aurélien vit Bérénice, il la trouva franchement laide”	Aragon
“Aujourd’hui maman est morte. Ou peut-être hier, je ne sais pas.”	Camus

Create a `citations` object that uses a suitable data structure in `Python` to:

1. Easily retrieve all quotations associated with an author. Create a `citation_aragon` object that tests the validity of your approach with the Aragon example.
2. For each quotation, count the number of words and the number of unique words. Create a `stats_phrases` object that lists these properties. Test on La Fontaine quotations.

Here are the quotes, in bulk, to get you started.

In this cell, create the appropriate data structure for this exercise.

i Hint 1

What data structure would enable this type of code?

```
citations.get("La Fontaine")
```

i Hint 2

What data structure could make an object look like

```
citations
```

i Hint 3

A dictionary may help. If you are still struggling to understand why and how, go back to the dedicated *notebook* ⬆️.

```
<IPython.core.display.HTML object>
```

For question 1, you can use this cell.

For question 2, you can use this cell.

Hint 1

How do you iterate over the keys and values of your `stats_phrases` dict?

Hint 2

Adopt this general structure :

```
for auteur, liste_citations in citations.items():
    # do something
```

How to have the length of a sentence (sequence of words) and single words in it?

Hint 3

How can I get the length of a sentence (sequence of words) and the single words in it?

1. A sentence (string) can be broken down into words (list) using the `split()` method.
2. A `set` can be used to deduplicate a list

Hint 4

You should get this type of result:

```
stats_phrases
```

```
<IPython.core.display.HTML object>
```

Warning

Ideally, you should pay attention to punctuation.

But we have not yet learned how to make refined substitutions in strings - that will come in another chapter.

1.2 Exercise 2

Advent of Code challenges are excellent practical problems for learning algorithms. With one problem per day of increasing difficulty between 1 December and Christmas Day, you will quickly become comfortable with the many data structures offered by Python.

The next exercise proposes solving the first part of day 1 (year 2022) using only basic Python.

Warning

It would be appropriate to use Numpy for the next problem. But the purpose of the next chapter is to show how this package simplifies basic numerical operations.

Here are the objects we will need for this exercise:

Exercise 2

This exercise aims to guide you in solving the problem on this page (Advent of Code 2022, day 1, part one). You will need to find a more suitable data structure than the initial one.

1. Read the instructions.
2. Working step by step, try to find the number of calories carried by each elf in the example.
3. Deduce which elf is carrying the most calories. Check that you have obtained the number of calories mentioned in the instructions for the example.
4. Generalize to solve the problem.

You can write your code for question 2 in this cell:

Hint 1

We will need to iterate over each value of `elves_example`: how can we transform the character string into a more manageable object?

Hint 2

Be careful with the type of object you will obtain. Don't forget to convert it to a numeric format if you want to perform arithmetic operations.

Hint 3

You should end up with the list

```
calories_example
```

```
<IPython.core.display.HTML object>
```

You can write your code for question 3 in this cell:

Finally, after generalizing, you should find the following values:

```
Elf number 217 carries the most calories (72602 calories).
```

1.3 Exercise 3: strings

This is the final pattern you should end up in question 1 and 2

```
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
```

Exercise 3

1. Using the `format` method, try to obtain the expected result.
2. Do the same with *f-strings*.

3. Modify the message to create a monthly income rounded to 1 digit. Use a space as the thousands separator rather than the default separator (the comma).
4. Here is a fictitious path `C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt` to a file. Make a correct print of it.

In the first part of this exercise, you should have these results.

```
Question 1:
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
Question 2:
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
Question 3:
On 17 March 2022  Nadia (34) reported an income of $2 777.8.
```

In the second part of the exercise, if you use the wrong type of *string*, you should get an error:

```
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated
\UXXXXXXXX escape (3077220908.py, line 2)
  ␛[36mCell␛[39m␛[36m ␛[39m␛[32mIn[14]␛[39m␛[32m, line 2␛[39m
␛[31m    ␛[39m␛[31mchemin_bad = "C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt"␛[39m
                ^
␛[31mSyntaxError␛[39m␛[31m:␛[39m (unicode error) 'unicodeescape' codec can't decode bytes in
position 2-3: truncated \UXXXXXXXX escape
```

Slashes often cause problems. With raw strings, you should not encounter this problem:

```
File can be found here: C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt
```

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```toml
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
```

```
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.